Python for automation

Write Python scripts that handle files, processes, HTTP APIs and cloud SDKs, with tests and a packaged command-line tool.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Python basics and virtual environments (15 min)
2. Files, processes and error handling (15 min)
3. HTTP, REST APIs and JSON (15 min)
4. Cloud SDKs: boto3 and the Azure SDK (16 min)
5. Testing and packaging a command-line tool (15 min)
6. Concurrency for operations: threads, asyncio and subprocesses (14 min)
7. Writing command-line tools in Python (13 min)
8. Logging for scripts and services (12 min)
9. Working with YAML, CSV and configuration files (12 min)
10. Debugging Python scripts and services (15 min)
11. SRE automation: AWS with boto3, safely (13 min)
12. SRE automation: Kubernetes client scripts and reports (12 min)
13. SRE automation: retries, backoff, timeouts and idempotency (12 min)
14. SRE automation: parsing logs and building incident timelines (12 min)
15. SRE automation: chatops, alert handlers and webhooks (12 min)
16. SRE automation: safe bulk operations and audit trails (12 min)
Cheat sheet

Lesson 1. Python basics and virtual environments

15 minutes. Write readable Python for automation, keep each project's packages isolated in a virtual environment, and run scripts the same way everywhere.

Bash is perfect for gluing commands together. Once a script needs data structures, error handling, JSON or an API call, Python is the better tool. Almost every cloud SDK, Ansible module and monitoring plugin you will meet is written in it, so a working knowledge pays off quickly.

What you will learn

- The small set of Python features that cover most automation work
- Why every project gets its own virtual environment
- How to install and pin dependencies with pip
- How to structure a script so it can be run and imported

The core you actually use

Python is indentation-sensitive: four spaces define a block. These building blocks show up in nearly every ops script.

```
servers = ["web-1", "web-2", "db-1"]            # list
limits = {"cpu": 80, "memory": 90}               # dict
name, port = "web-1", 8080                       # tuple unpacking

web = [s for s in servers if s.startswith("web")]   # list comprehension
print(f"{name}:{port} has {len(web)} web peers")    # f-string

def over_limit(metric: str, value: float) -> bool:
    """Return True when value is above the configured limit."""
    return value > limits.get(metric, 100)

for metric, value in {"cpu": 91.5, "memory": 40}.items():
    if over_limit(metric, value):
        print(f"ALERT {metric}={value}")
```

Three habits make scripts safer. Use **f-strings** for formatting, use `dict.get(key, default)` instead of indexing when a key may be missing, and add short **type hints** so the next reader knows what a function expects.

Virtual environments

Python packages install into a Python interpreter. If every project installs into the system Python, two projects needing different versions of the same library will break each other, and an OS upgrade can break your tools. A **virtual environment** is a private folder holding its own interpreter links and packages.

```
$ python3 -m venv .venv
$ source .venv/bin/activate
$ python -m pip install --upgrade pip
$ python -m pip install requests==2.32.3
$ python -m pip freeze > requirements.txt
$ deactivate
```

Create the environment once per project, activate it in each new shell, and keep `.venv/` out of version control and out of container images built from your code. The `requirements.txt` file records exact versions so a teammate or a pipeline can reproduce the same set:

```
$ python3 -m venv .venv && . .venv/bin/activate
$ python -m pip install -r requirements.txt
$ python -m pip list --outdated
```

MARGA by EknathaLabs marga.eknathalabs.com

Always call pip as `python -m pip`. That guarantees the pip belongs to the interpreter you think you are using.

A script that can also be imported

```
import sys


def main(argv: list[str]) -> int:
    if len(argv) != 2:
        print("usage: check.py HOSTNAME", file=sys.stderr)
        return 2
    print(f"checking {argv[1]}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main(sys.argv))
```

The `if __name__ == "__main__"` guard means the file runs when executed but does nothing when another file imports a function from it. Returning an integer and passing it to `SystemExit` gives the shell a proper exit code, which pipelines rely on.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Python basics and virtual environments fits together

Part	What it does	What goes wrong
The core you actually use	Python is indentation-sensitive: four spaces define a block.	Using mutable default arguments such as def f(items=[]).
Virtual environments	Python packages install into a Python interpreter.	Installing packages into the system Python with sudo pip.
A script that can also be imported	The if __name__ == "__main__" guard means the file runs when executed but does nothing when another file imports a function from it.	Forgetting to pin versions, so a new release breaks a working script.

Flow: The core you actually use to Virtual environments / Virtual environments to A script that can also be imported

Incident: the upgrade that broke the backup script

> **Incident:** *After a routine OS update, the nightly backup script fails with* `ModuleNotFoundError: No module named 'boto3'`. *The script ran for two years. Nobody remembers installing boto3, and a quick* `pip install` *as root makes the error go away until the next update.*

The script depended on a package that had been installed into the system Python, and the update replaced that interpreter. The durable fix is to give the script its own environment and record its dependencies:

```
$ python3 -m venv /opt/backup/.venv
$ /opt/backup/.venv/bin/python -m pip install -r /opt/backup/requirements.txt
$ /opt/backup/.venv/bin/python /opt/backup/backup.py
```

Call the environment's interpreter by absolute path in cron or systemd, so no activation step is needed. The script is now independent of the OS Python, and a rebuilt server gets the same versions from `requirements.txt`.

Common mistakes

- Installing packages into the system Python with `sudo pip`
- Forgetting to pin versions, so a new release breaks a working script
- Using mutable default arguments such as `def f(items=[])`

MARGA by EknathaLabs marga.eknathalabs.com

- Catching every exception with a bare `except:` and hiding the real failure
- Mixing tabs and spaces for indentation

Check yourself

Why create a virtual environment per project?

Answer: It isolates each project's packages so versions do not conflict

What does `python -m pip freeze > requirements.txt` produce?

Answer: A list of installed packages with exact versions, usable to recreate the environment

What does the `if __name__ == "__main__"` guard do?

Answer: Runs the code only when the file is executed, not when it is imported

How should cron run a script that needs a virtual environment?

Answer: Call the environment's interpreter by absolute path, such as .venv/bin/python

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Files, processes and error handling

15 minutes. Read and write files safely with pathlib, run other programs with subprocess, handle failures explicitly and log what happened.

Most automation boils down to three jobs: read or write files, run another program, and decide what to do when something fails. Python's standard library covers all three without installing anything.

What you will learn

- Reading and writing files with `pathlib` and context managers
- Running commands with `subprocess.run` safely and checking the result
- Handling exceptions deliberately and exiting with useful codes
- Logging instead of scattering `print` calls

Files with pathlib

`pathlib.Path` treats paths as objects, so you avoid string concatenation and platform differences.

```
from pathlib import Path

config = Path("/etc/myapp/app.conf")
if not config.exists():
    raise SystemExit(f"missing config: {config}")

text = config.read_text(encoding="utf-8")
lines = [l.strip() for l in text.splitlines() if l and not l.startswith("#")]

out = Path("/var/tmp/report.txt")
out.parent.mkdir(parents=True, exist_ok=True)
with out.open("w", encoding="utf-8") as fh:
    for line in lines:
        fh.write(line + "\n")

for log in Path("/var/log/myapp").glob("*.log"):
    print(log.name, log.stat().st_size)
```

The `with` block closes the file even when an error occurs. For configuration files in JSON or YAML, parse them with `json.loads` or a YAML library instead of splitting lines yourself.

Running programs

`subprocess.run` is the safe default. Pass the command as a **list**, not a string, so the shell never interprets it.

```
import subprocess

result = subprocess.run(
    ["systemctl", "is-active", "nginx"],
    capture_output=True, text=True, timeout=10,
)
print(result.returncode, result.stdout.strip())

# fail loudly when a command must succeed
subprocess.run(["tar", "-czf", "/tmp/site.tgz", "/var/www"], check=True)
```

`check=True` raises `CalledProcessError` on a non-zero exit. `timeout` stops a hung command from hanging your script. Avoid `shell=True` with any value that comes from a user, a file or an API, because it enables command injection.

Exceptions you expect

Catch the specific errors you can handle and let everything else stop the script with a traceback.

```
import subprocess
import sys

def restart(service: str) -> bool:
```

MARGA by EknathaLabs marga.eknathalabs.com

```
    try:
        subprocess.run(["systemctl", "restart", service], check=True, timeout=30)
    except subprocess.CalledProcessError as exc:
        print(f"restart failed, exit code {exc.returncode}", file=sys.stderr)
        return False
    except subprocess.TimeoutExpired:
        print("restart timed out", file=sys.stderr)
        return False
    return True
```

Logging

`logging` gives timestamps and levels, and can be redirected to files or the journal without changing the code.

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)
log = logging.getLogger("cleanup")
log.info("removing files older than %d days", 14)
log.warning("directory %s is almost full", "/var/tmp")
```

Use `log.exception("message")` inside an `except` block to record the traceback.

```
$ python3 cleanup.py
2025-06-02 03:00:01,114 INFO removing files older than 14 days
2025-06-02 03:00:01,201 WARNING directory /var/tmp is almost full
$ echo $?
0
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Files, processes and error handling fits together

Part	What it does	What goes wrong
Files with pathlib	pathlib.Path treats paths as objects, so you avoid string concatenation and platform differences.	Using print for everything instead of logging with levels.
Running programs	subprocess.run is the safe default. Pass the command as a list, not a string, so the shell never interprets it.	Using shell=True with interpolated values.
Exceptions you expect	Catch the specific errors you can handle and let everything else stop the script with a traceback.	Swallowing exceptions so the script reports success when it failed.
Logging	logging gives timestamps and levels, and can be redirected to files or the journal without changing the code.	Opening files without a context manager, leaving them unclosed.

Flow: Files with pathlib to Running programs / Running programs to Exceptions you expect / Exceptions you expect to Logging

Incident: the cleanup that deleted too much

> **Incident:** *A cleanup script runs* `subprocess.run(f"rm -rf {base}/{name}", shell=True)`*. One night* `name` *comes back empty from a config file, and the command becomes* `rm -rf /data/`*. Half of the shared volume is deleted before anyone notices.*

Three defects combined: a shell string, an unvalidated value and no dry run. The safer version validates input, uses `pathlib` and refuses to act on anything outside the intended directory:

```
import shutil
from pathlib import Path

BASE = Path("/data/cache").resolve()
```

```
def remove(name: str, dry_run: bool = True) -> None:
    if not name or "/" in name:
        raise ValueError(f"refusing unsafe name: {name!r}")
    target = (BASE / name).resolve()
    if BASE not in target.parents:
        raise ValueError(f"{target} is outside {BASE}")
    print(("would remove " if dry_run else "removing ") + str(target))
    if not dry_run:
        shutil.rmtree(target)
```

Default to a dry run, print what would happen, and require an explicit flag for the destructive step.

Predict the result

Common mistakes

- Using `shell=True` with interpolated values
- Opening files without a context manager, leaving them unclosed
- Forgetting `encoding="utf-8"`, which breaks on non-English systems
- Using `print` for everything instead of logging with levels
- Swallowing exceptions so the script reports success when it failed

Check yourself

Why pass a list to subprocess.run instead of a single string?

Answer: The shell never interprets the arguments, which prevents injection and quoting bugs

What does check=True do?

Answer: Raises CalledProcessError when the command exits non-zero

What is the benefit of a with statement when opening a file?

Answer: The file is closed automatically, even if an error occurs

Why default a destructive script to dry-run mode?

Answer: Operators can review what would change before anything is deleted

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. HTTP, REST APIs and JSON

15 minutes. Call REST APIs with requests, parse and build JSON, handle timeouts, retries and pagination, and keep tokens out of your code.

Cloud providers, ticketing systems, chat tools and monitoring platforms all expose REST APIs. Calling them from Python turns a repetitive console task into a script you can schedule and review.

What you will learn

- Making GET and POST requests with the `requests` library
- Working with JSON in both directions
- Timeouts, status codes, retries and pagination
- Reading secrets from the environment

A first request

```
import requests

resp = requests.get(
    "https://api.example.com/v1/servers",
    headers={"Accept": "application/json"},
    params={"status": "running", "limit": 50},
    timeout=10,
)
resp.raise_for_status()
for server in resp.json()["items"]:
    print(server["id"], server["name"])
```

Three details matter. **Always set a timeout**: without one, `requests` can wait forever. **Call** `raise_for_status()` so 4xx and 5xx responses become exceptions instead of silently parsed error pages. **Use** `params=` rather than building query strings by hand, because it handles encoding.

Sending JSON

```
import os
import requests

token = os.environ["API_TOKEN"]          # never hard-code tokens
payload = {"name": "web-3", "size": "small", "tags": ["prod", "web"]}

resp = requests.post(
    "https://api.example.com/v1/servers",
    json=payload,                        # sets the header and encodes for you
    headers={"Authorization": f"Bearer {token}"},
    timeout=10,
)
if resp.status_code == 409:
    print("already exists")
else:
    resp.raise_for_status()
    print("created", resp.json()["id"])
```

Reading `os.environ["API_TOKEN"]` fails immediately with a `KeyError` if the variable is missing, which is better than sending an empty header. Never commit tokens, print them or include them in error messages.

Pagination and retries

APIs return data in pages. A reusable session with retries handles brief outages and rate limits.

```
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

def make_session() -> requests.Session:
    retry = Retry(total=4, backoff_factor=0.5,
                  status_forcelist=(429, 500, 502, 503, 504),
```

MARGA by EknathaLabs marga.eknathalabs.com

```
                allowed_methods=("GET", "PUT", "DELETE"))
    s = requests.Session()
    s.mount("https://", HTTPAdapter(max_retries=retry))
    return s

def all_servers(session: requests.Session):
    url = "https://api.example.com/v1/servers"
    while url:
        r = session.get(url, timeout=10)
        r.raise_for_status()
        body = r.json()
        yield from body["items"]
        url = body.get("next")
```

Do not retry non-idempotent `POST` requests blindly: a retry after a timeout could create a duplicate. Honour `Retry-After` headers and use exponential backoff so you do not hammer a service that is already struggling.

JSON basics

```
import json

raw = '{"name": "web-1", "cpu": 0.42, "tags": ["prod"]}'
data = json.loads(raw)                  # string -> dict
data["cpu"] += 0.1
print(json.dumps(data, indent=2, sort_keys=True))

with open("servers.json", "w", encoding="utf-8") as fh:
    json.dump(data, fh, indent=2)
```

```
$ python3 list_servers.py | head -3
srv-001 web-1
srv-002 web-2
srv-003 db-1
```

Incident: the script that hung the deployment

> **Incident:** *A deployment step calls a status API with `requests.get(url)`. The API's load balancer stalls, the connection never closes, and the pipeline job hangs for six hours until a runner timeout kills it. The release is delayed and the runner pool is exhausted.*

`requests` has no default timeout. The fix is a single argument, and a retry policy for the transient case:

```
resp = requests.get(url, timeout=(3.05, 10))   # connect timeout, read timeout
```

Wrap the call in the session from the previous section so a short outage is retried a few times with backoff, then fails the job quickly with a clear error. A fast failure with a message is always better than a silent hang.

Diagram: A reliable API call from a script

Part	What it does	What goes wrong
Build the request	URL, headers, token from the environment and a timeout.	No timeout means a hung server hangs your script forever.
Send with retries	A session retries on 429 and 5xx with backoff.	Retrying non-idempotent calls can create duplicates.
Check the status	Call raise_for_status so 4xx and 5xx become errors.	Ignoring status parses an error page as data.
Parse JSON	Use response.json() and read fields with get.	A missing key raises KeyError or silently gives None.
Use or log the result	Return data or log the failure with the request id.	Without the id you cannot find the matching server log.

Flow: Build the request to Send with retries / Send with retries to Check the status / Send with retries to Parse JSON / Check the status to Use or log the result / Parse JSON to Use or log the result

Common mistakes

MARGA by EknathaLabs
marga.eknathalabs.com

- Omitting `timeout`, which allows indefinite hangs
- Parsing the body without checking the status code
- Building URLs and query strings with string concatenation
- Logging full request headers, which leaks tokens
- Retrying `POST` requests that are not idempotent

Check yourself

What happens if you call requests.get(url) without a timeout?

Answer: It can wait indefinitely if the server never responds

What does resp.raise_for_status() do?

Answer: Raises an exception for 4xx and 5xx responses

Where should an API token be stored for a script?

Answer: In an environment variable or secret store read at runtime

Why be careful retrying a POST request?

Answer: A retry after a timeout might create a duplicate resource

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Cloud SDKs: boto3 and the Azure SDK

16 minutes. Automate AWS with boto3 and Azure with its Python SDK, authenticate without hard-coded keys, paginate results and write idempotent scripts.

The cloud consoles and CLIs are fine for exploring, but repeatable work belongs in code. Both AWS and Azure publish official Python SDKs, and the patterns you learn for one transfer directly to the other.

What you will learn

- Authenticating with roles and default credential chains
- Listing and filtering resources with boto3, including pagination
- The same tasks with the Azure SDK
- Writing scripts that are safe to run twice

Authentication first

Never put access keys in a script. On a developer machine use SSO or a profile. On a server or pipeline use an **instance role**, a **workload identity** or an assumed role, so the SDK finds short-lived credentials by itself.

```
$ aws sso login --profile dev
$ export AWS_PROFILE=dev
$ python3 -c "import boto3; print(boto3.client('sts').get_caller_identity()['Arn'])"
arn:aws:sts::123456789012:assumed-role/Developer/reddy
```

Always print the identity first in any script that changes things. Running the right code against the wrong account is a classic way to cause an outage.

boto3 essentials

```
import boto3

ec2 = boto3.client("ec2", region_name="eu-west-1")

paginator = ec2.get_paginator("describe_instances")
pages = paginator.paginate(
    Filters=[{"Name": "tag:Env", "Values": ["prod"]},
             {"Name": "instance-state-name", "Values": ["running"]}]
)
for page in pages:
    for reservation in page["Reservations"]:
        for inst in reservation["Instances"]:
            name = next((t["Value"] for t in inst.get("Tags", []) if t["Key"] == "Name"), "-")
            print(inst["InstanceId"], inst["InstanceType"], name)
```

API calls return at most one page of results. A **paginator** hides the continuation tokens; without it you silently process only the first page. Filter on the server with `Filters=` instead of downloading everything.

A safe, repeatable task

Scripts that change infrastructure should be **idempotent**: running them twice has the same result as once.

```
import boto3
from botocore.exceptions import ClientError

s3 = boto3.client("s3")

def ensure_bucket(name: str, region: str) -> None:
    try:
        s3.head_bucket(Bucket=name)
        print("exists:", name)
    except ClientError as exc:
        if exc.response["Error"]["Code"] not in ("404", "NoSuchBucket"):
```

MARGA by EknathaLabs marga.eknathalabs.com

```
            raise
        s3.create_bucket(Bucket=name,
                         CreateBucketConfiguration={"LocationConstraint": region})
        s3.put_public_access_block(
            Bucket=name,
            PublicAccessBlockConfiguration={
                "BlockPublicAcls": True, "IgnorePublicAcls": True,
                "BlockPublicPolicy": True, "RestrictPublicBuckets": True})
        print("created:", name)

ensure_bucket("acme-reports-eu", "eu-west-1")
```

Check first, create only if missing, and apply the secure settings in the same script.

The Azure equivalent

```
from azure.identity import DefaultAzureCredential
from azure.mgmt.compute import ComputeManagementClient

cred = DefaultAzureCredential()          # CLI login, managed identity, or env vars
compute = ComputeManagementClient(cred, subscription_id="00000000-0000-0000-0000-000000000000")

for vm in compute.virtual_machines.list_all():
    print(vm.name, vm.location, vm.hardware_profile.vm_size)
```

`DefaultAzureCredential` tries several sources in order, so the same code works on a laptop after `az login` and in Azure with a managed identity. Install the packages in your virtual environment:

```
$ python -m pip install boto3 azure-identity azure-mgmt-compute
$ python -m pip freeze | grep -E "boto3|azure"
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Cloud SDKs: boto3 and the Azure SDK fits together

Part	What it does	What goes wrong
Authentication first	Never put access keys in a script. On a developer machine use SSO or a profile.	Hard-coding access keys in scripts or committing them.
boto3 essentials	API calls return at most one page of results.	Ignoring pagination and processing only the first page.
A safe, repeatable task	Scripts that change infrastructure should be idempotent: running them twice has the same result as once.	Writing scripts that fail when a resource already exists.
The Azure equivalent	DefaultAzureCredential tries several sources in order, so the same code works on a laptop after az login and in Azure with a managed identity.	Running without checking the account or subscription.

Flow: Authentication first to boto3 essentials / boto3 essentials to A safe, repeatable task / A safe, repeatable task to The Azure equivalent

Incident: the script that ran in the wrong account

> **Incident:** *An engineer runs a cleanup script that deletes untagged volumes. Their shell still has the production profile exported from the previous day. Forty production volumes are removed before the first alert fires.*

Credentials came from ambient shell state, and the script never confirmed where it was running. Add three guards:

```
import sys
import boto3

EXPECTED_ACCOUNT = "111122223333"
account = boto3.client("sts").get_caller_identity()["Account"]
if account != EXPECTED_ACCOUNT:
    sys.exit(f"refusing to run in account {account}")
```

Pair this with a dry-run default, a tag-based allow list and snapshots before deletion. Production permissions should also be separate roles that require an explicit assume-role step.

Common mistakes

- Hard-coding access keys in scripts or committing them
- Ignoring pagination and processing only the first page
- Running without checking the account or subscription
- Writing scripts that fail when a resource already exists
- Catching `ClientError` broadly and hiding real permission problems

Check yourself

What is the best way for a script on an EC2 instance to authenticate?

Answer: Use the instance role through the default credential chain

Why use a paginator?

Answer: API responses are split into pages and you would otherwise see only the first

What does idempotent mean for an automation script?

Answer: Running it repeatedly has the same result as running it once

What does DefaultAzureCredential do?

Answer: Tries several credential sources in order, such as CLI login or managed identity

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Testing and packaging a command-line tool

15 minutes. Build a real CLI with argparse, test it with pytest including mocks, and package it so teammates can install it with one command.

A script becomes a tool when it has arguments, tests and a way to install it. These habits turn a one-off helper into something your team trusts at 3 a.m.

What you will learn

- Building a command-line interface with `argparse`
- Writing tests with `pytest` and mocking external calls
- Packaging with `pyproject.toml` and installing the tool
- Running linters and tests in a pipeline

A small CLI

```
import argparse
import sys


def parse_args(argv=None):
    p = argparse.ArgumentParser(prog="diskwatch", description="Warn when disks are nearly full")
    p.add_argument("paths", nargs="+", help="mount points to check")
    p.add_argument("--threshold", type=int, default=85, help="percent used that triggers a warning \
    ")
    p.add_argument("--json", action="store_true", help="print machine-readable output")
    return p.parse_args(argv)


def percent_used(path: str) -> int:
    import shutil
    usage = shutil.disk_usage(path)
    return round(usage.used / usage.total * 100)


def main(argv=None) -> int:
    args = parse_args(argv)
    bad = [(p, percent_used(p)) for p in args.paths if percent_used(p) >= args.threshold]
    for path, pct in bad:
        print(f"WARN {path} is {pct}% full")
    return 1 if bad else 0


if __name__ == "__main__":
    sys.exit(main())
```

`argparse` generates `--help`, validates types and gives clear errors for free. Returning a non-zero exit code when a check fails lets cron, monitoring and pipelines react.

Testing with pytest

Keep logic in small functions, then test them. Replace anything slow or dangerous, such as disks or HTTP calls, with a **mock**.

```
from unittest import mock
import diskwatch


def test_warns_when_over_threshold(capsys):
    with mock.patch("diskwatch.percent_used", return_value=91):
        code = diskwatch.main(["/data", "--threshold", "85"])
    assert code == 1
    assert "WARN /data is 91% full" in capsys.readouterr().out

def test_ok_when_under_threshold():
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
    with mock.patch("diskwatch.percent_used", return_value=40):
        assert diskwatch.main(["/data"]) == 0


def test_requires_a_path():
    import pytest
    with pytest.raises(SystemExit):
        diskwatch.parse_args([])
```

```
$ python -m pip install pytest
$ python -m pytest -q
...                                                                  [100%]
3 passed in 0.04s
```

Test the failure paths as hard as the happy path: that is where operations scripts hurt.

Packaging

A `pyproject.toml` describes the project. Installing it creates a real command on the PATH.

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "diskwatch"
version = "0.1.0"
requires-python = ">=3.9"

[project.scripts]
diskwatch = "diskwatch:main"
```

The `[project.scripts]` table maps the command name to the function, so `pip install .` creates the `diskwatch` command. Install it into a virtual environment, or with `pipx` so it stays isolated:

```
$ python -m pip install .
$ diskwatch /var --threshold 80
$ diskwatch --help
```

Checks in a pipeline

```
test:
  image: python:3.12-slim
  script:
    - python -m pip install -r requirements.txt pytest ruff
    - ruff check .
    - python -m pytest -q
```

Linting catches mistakes before runtime, and the test job blocks a merge when behaviour breaks.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Testing and packaging a command-line tool fits together

Part	What it does	What goes wrong
A small CLI	argparse generates --help, validates types and gives clear errors for free.	Putting all logic under if __name__ == "__main__" where it cannot be tested.
Testing with pytest	Keep logic in small functions, then test them.	Testing against real servers or production data.
Packaging	A pyproject.toml describes the project. Installing it creates a real command on the PATH.	Pinning nothing, so a new dependency release breaks the build.
Checks in a pipeline	Linting catches mistakes before runtime, and the test job blocks a merge when behaviour breaks.	Checks in a pipeline Linting catches mistakes before runtime, and the test job blocks a merge when behaviour breaks.

Flow: A small CLI to Testing with pytest / Testing with pytest to Packaging / Packaging to Checks in a pipeline

MARGA by EknathaLabs marga.eknathalabs.com

Incident: the quiet change to a shared script

> **Incident:** *Someone improves a shared* `rotate_logs.py` *to compress files in place. Nobody tests it. The next night it raises an exception on a file name with spaces, stops halfway, and leaves partial archives. Disk fills up on three hosts by morning.*

With a test suite the problem is found in seconds: add a test with an awkward file name that fails, fix the code, and keep the test forever.

```
from pathlib import Path
import rotate_logs

def test_handles_spaces(tmp_path: Path):
    f = tmp_path / "app log 1.txt"
    f.write_text("data")
    rotate_logs.compress(f)
    assert (tmp_path / "app log 1.txt.gz").exists()
```

`tmp_path` gives each test a clean temporary directory, so tests never touch real data.

Common mistakes

- Putting all logic under `if __name__ == "__main__"` where it cannot be tested
- Testing against real servers or production data
- Not returning meaningful exit codes
- Pinning nothing, so a new dependency release breaks the build
- Skipping tests for the error paths

Check yourself

Why return an exit code from a CLI tool?

Answer: Schedulers and pipelines use it to decide whether the run succeeded

What is a mock used for in tests?

Answer: To replace slow or risky dependencies such as network or disk calls

What does pytest's tmp_path fixture provide?

Answer: A fresh temporary directory for each test

How does packaging make a script available as a command?

Answer: A scripts entry point in pyproject.toml makes pip install create the command

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Concurrency for operations: threads, asyncio and subprocesses

14 minutes. Run many network calls or commands at once with thread pools, asyncio and subprocess, with timeouts, limits and error handling that keep operations scripts fast and safe.

Operations scripts spend most of their time waiting: for an API, for a remote host, for a command. Running a hundred checks one after another can take an hour; running them concurrently can take a minute. Python gives you three tools, and choosing the right one and putting limits on it matters more than speed.

What you will learn

- Why waiting work benefits from concurrency and CPU work does not
- Thread pools with `concurrent.futures`
- `asyncio` for many network calls
- Running commands with `subprocess` and timeouts
- Limiting concurrency and handling failures

Which tool

Work	Tool
Many blocking calls (HTTP, SSH, SDK)	`ThreadPoolExecutor`
Very many network connections, async libraries available	`asyncio`
Heavy CPU work	`ProcessPoolExecutor` or separate processes
External commands	`subprocess.run` with `timeout`

The global interpreter lock stops threads from running Python code in parallel, but threads waiting on I/O release it, so thread pools work well for operations tasks.

A thread pool with limits

```
import requests
from concurrent.futures import ThreadPoolExecutor, as_completed

def check(url):
    r = requests.get(url, timeout=(3, 10))
    return url, r.status_code

urls = [f"https://svc{i}.example.com/health" for i in range(100)]
failed = []
with ThreadPoolExecutor(max_workers=20) as pool:
    futures = {pool.submit(check, u): u for u in urls}
    for fut in as_completed(futures):
        try:
            url, status = fut.result()
            if status != 200:
                failed.append((url, status))
        except Exception as exc:
            failed.append((futures[fut], repr(exc)))
print(f"{len(failed)} problems")
```

`max_workers` is the limit. Without one, a hundred workers may overload the target or exhaust your file handles. Always set a timeout on each call and collect failures instead of stopping at the first.

asyncio

```
import asyncio, aiohttp

async def check(session, sem, url):
    async with sem:
        async with session.get(url, timeout=aiohttp.ClientTimeout(total=10)) as r:
            return url, r.status

async def main(urls):
    sem = asyncio.Semaphore(50)
    async with aiohttp.ClientSession() as session:
        return await asyncio.gather(*(check(session, sem, u) for u in urls), return_exceptions=Tru \
    e)

results = asyncio.run(main(urls))
```

Everything in the call chain must be async, and one blocking call stalls the whole loop. `return_exceptions=True` keeps one failure from cancelling the rest. A `Semaphore` caps concurrent calls.

Running commands

```
import subprocess
p = subprocess.run(["df", "-h", "/var"], capture_output=True, text=True, timeout=10, check=False)
if p.returncode != 0:
    raise RuntimeError(p.stderr.strip())
```

Pass a list, not a string with `shell=True`, so that input cannot inject commands. Always set `timeout`.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Concurrency for operations: threads, asyncio and... fits together

Part	What it does	What goes wrong
Which tool	The global interpreter lock stops threads from running Python code in parallel, but threads waiting on I/O release it, so thread pools work well for operations tasks.	Using shell=True with input from outside.
A thread pool with limits	max_workers is the limit. Without one, a hundred workers may overload the target or exhaust your file handles.	Stopping the whole run on the first error.
asyncio	Everything in the call chain must be async, and one blocking call stalls the whole loop.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Running commands	Pass a list, not a string with shell=True, so that input cannot inject commands.	Unlimited concurrency.

Flow: Which tool to A thread pool with limits / A thread pool with limits to asyncio / asyncio to Running commands

Incident: the audit that took down the inventory API

> **Incident:** *A new audit script queries every instance's details at once and the internal inventory API starts returning errors for everyone.*

```
$ python audit.py
... 3000 requests in 4 seconds
429 Too Many Requests
```

The script started a thread for each item with no limit. The fix used a pool of 10 workers, a retry with exponential backoff on 429 responses, and a result file written as items finished so a failure did not lose the work.

Common mistakes

- Unlimited concurrency

- No timeouts
- Stopping the whole run on the first error
- Using `shell=True` with input from outside

Check yourself

Why do threads help for network-bound scripts despite the GIL?

Answer: Threads release the GIL while they wait for I/O

What does max_workers control?

Answer: How many tasks run at the same time

How do you stop one failed task from cancelling the rest in asyncio.gather?

Answer: Pass return_exceptions=True

Why pass a list to subprocess.run instead of a string with shell=True?

Answer: It avoids shell injection from untrusted values

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Writing command-line tools in Python

13 minutes. Build a clear, testable command-line tool with argparse, structured output, logging, exit codes and configuration from the environment, ready to be packaged and shared.

Many operations scripts start as twenty lines and grow into tools that a whole team runs. A good command-line tool is predictable: it has help text, validates input, logs sensibly, returns the right exit code and can be tested. Python's standard library gives you everything you need to get there.

What you will learn

- Defining arguments and subcommands with `argparse`
- Reading configuration from options, environment variables and files
- Human and machine-readable output
- Logging, exit codes and error handling
- Testing a command-line tool and packaging an entry point

Arguments with argparse

```
import argparse, logging, sys

def build_parser():
    p = argparse.ArgumentParser(prog="svcctl", description="Inspect services")
    p.add_argument("-v", "--verbose", action="store_true", help="show debug logs")
    sub = p.add_subparsers(dest="cmd", required=True)
    s = sub.add_parser("status", help="show service status")
    s.add_argument("name")
    s.add_argument("--json", action="store_true", help="machine-readable output")
    s.add_argument("--timeout", type=float, default=5.0)
    return p

def main(argv=None):
    args = build_parser().parse_args(argv)
    logging.basicConfig(level=logging.DEBUG if args.verbose else logging.WARNING,
                        format="%(levelname)s %(message)s", stream=sys.stderr)
    try:
        return run(args)
    except KeyboardInterrupt:
        return 130
    except Exception as exc:
        logging.error("%s", exc)
        return 1

if __name__ == "__main__":
    sys.exit(main())
```

`argparse` builds `--help`, checks types and rejects unknown options with exit code 2. Taking `argv` as a parameter makes `main` easy to test.

Configuration order

Use this order of precedence: command-line option, then environment variable, then a config file, then a default. Document it. Read secrets from the environment or a secret store, never from a flag (flags show up in process lists and shell history).

```
import os
token = os.environ.get("SVC_TOKEN")
if not token:
    raise SystemExit("SVC_TOKEN is not set")
```

MARGA by EknathaLabs marga.eknathalabs.com

Output

Print results to standard output and logs and errors to standard error. Offer `--json` for scripts and a readable table for people. Never mix log lines into JSON output. Keep field names stable, because other tools will depend on them.

Exit codes

0 means success, 1 a general failure, 2 a usage error (the `argparse` default) and 130 an interrupt. If your tool is used in checks, define extra codes (for example 3 for "degraded") and document them.

Testing

```
def test_status_json(capsys, monkeypatch):
    monkeypatch.setattr("svcctl.fetch", lambda name, timeout: {"name": name, "ok": True})
    code = main(["status", "web", "--json"])
    out = capsys.readouterr().out
    assert code == 0 and '"ok": true' in out
```

Test the function, not the process. Use `subprocess` only for a small number of end-to-end checks.

Packaging

In `pyproject.toml`, add `[project.scripts]` with `svcctl = "svcctl.cli:main"`. Installing the package (preferably with `pipx`) creates a `svcctl` command. Pin dependencies in a lock file and include a version flag (`--version`).

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Writing command-line tools in Python fits together

Part	What it does	What goes wrong
Arguments with argparse	argparse builds --help, checks types and rejects unknown options with exit code 2.	Printing logs on standard output.
Configuration order	Use this order of precedence: command-line option, then environment variable, then a config file, then a default.	Secrets passed as flags.
Output	Print results to standard output and logs and errors to standard error.	Never mix log lines into JSON output.
Exit codes	0 means success, 1 a general failure, 2 a usage error (the argparse default) and 130 an interrupt.	Unclear exit codes.
Testing	Test the function, not the process. Use subprocess only for a small number of end-to-end checks.	Code in if __name__ blocks that cannot be tested.
Packaging	In pyproject.toml, add [project.scripts] with svcctl = "svcctl.cli:main".	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Arguments with argparse to Configuration order / Configuration order to Output / Output to Exit codes / Exit codes to Testing / Testing to Packaging

Incident: the JSON that wasn't JSON

Incident: *A monitoring job that parses the tool's `--json` output starts failing after a release.*

```
$ svcctl status web --json | head -2
INFO connecting to web
{"name": "web", "ok": true}
```

A new log line went to standard output and broke the parser. The fix sent logs to standard error, added a test that parses the output as JSON, and documented that standard output contains only the result.

MARGA by EknathaLabs
marga.eknathalabs.com

Common mistakes

- Printing logs on standard output
- Secrets passed as flags
- Unclear exit codes
- Code in `if __name__` blocks that cannot be tested

Check yourself

What exit code does argparse use for bad usage?

Answer: 2

Why take argv as a parameter of main?

Answer: So tests can call main with a list of arguments

Where should log messages go in a tool that prints JSON results?

Answer: Standard error

Why avoid passing secrets as command-line flags?

Answer: They appear in process lists and shell history

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Logging for scripts and services

12 minutes. Replace print with the logging module: levels, formats, handlers, structured fields and logs that help during an incident.

A nightly script prints "done" and nothing else, so when it breaks nobody knows which step failed or how long it took. Using `print` for everything also mixes results with diagnostics, so you cannot send one to a file and the other to the screen. Python ships a logging module that solves this in a few lines.

What you will learn

- Levels: DEBUG, INFO, WARNING, ERROR, CRITICAL
- Configure format, destination and level once at start-up
- Log exceptions with tracebacks and useful context
- Structured (JSON) logs that tools can search

Basic setup

Configure logging once, in the main program, not in libraries. Every module then asks for a logger by name.

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s",
)
log = logging.getLogger(__name__)

log.info("starting export for %s", "2026-01-05")
log.warning("retrying page %d", 7)
log.error("export failed")
```

Use `%s` placeholders and pass values as arguments instead of building the string with an f-string. The text is only built if the message is actually emitted, which saves time on DEBUG lines in a tight loop.

Choose levels on purpose

Level	Use it for
DEBUG	Details you need only when diagnosing a problem
INFO	Normal milestones: started, finished, counts
WARNING	Something unexpected, but the work continues
ERROR	An operation failed
CRITICAL	The program cannot continue

Production usually runs at INFO. Make the level a setting (for example the `LOG_LEVEL` environment variable) so that you can raise it to DEBUG for one run without changing code. Avoid logging at ERROR for things that are normal, because people stop trusting the signal.

Log failures with context

`log.exception` records an error message together with the full traceback, and is meant to be used inside an `except` block.

```
try:
    push_report(day)
except requests.Timeout:
    log.exception("report push timed out day=%s", day)
    raise
```

Include the identifiers someone needs to find the case: the file name, the customer id, the request id, the attempt number. Never log secrets, tokens or full card numbers. A good error line answers three

MARGA by EknathaLabs
marga.eknathalabs.com

questions: what failed, for which item, and what happens next.

Files, rotation and JSON

Scripts that run under systemd or in a container should write to standard output and let the platform collect it. For a plain server, use a rotating file handler so the log cannot fill the disk.

```
from logging.handlers import RotatingFileHandler
handler = RotatingFileHandler("/var/log/report.log", maxBytes=5_000_000, backupCount=5)
logging.getLogger().addHandler(handler)
```

For log search tools, one JSON object per line is easier to query than free text. A small custom formatter or a library can produce lines such as `{"ts": "...", "level": "ERROR", "msg": "push failed", "day": "2026-01-05"}`. Keep the field names the same in every program.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Logging for scripts and services fits together

Part	What it does	What goes wrong
Basic setup	Configure logging once, in the main program, not in libraries.	Calling basicConfig inside a library, which fights the application settings.
Choose levels on purpose	Production usually runs at INFO. Make the level a setting (for example the LOG_LEVEL environment variable) so that you can raise it to DEBUG for one run without changing...	Catching an exception and logging nothing.
Log failures with context	log.exception records an error message together with the full traceback, and is meant to be used inside an except block.	Logging secrets or personal data.
Files, rotation and JSON	Scripts that run under systemd or in a container should write to standard output and let the platform collect it.	Using print for diagnostics and results at the same time.

Flow: Basic setup to Choose levels on purpose / Choose levels on purpose to Log failures with context / Log failures with context to Files, rotation and JSON

Incident: Done, but did it work?

Incident: *A nightly sync ended with the line "done", yet customers were missing from the next morning report.*

```
$ tail -3 /var/log/sync.out
fetching page 41
fetching page 42
done
$ grep -c ERROR /var/log/sync.out
0
```

The script caught every exception and printed nothing, so a timeout on page 38 was lost. After the change it logged a warning for each retry, an error with the page number when a page was skipped, and a final INFO line with counts (`fetched=412 skipped=1`). The scheduler marked the run as failed whenever skipped was above zero, so the next gap was reported within minutes.

Common mistakes

- Using print for diagnostics and results at the same time
- Catching an exception and logging nothing
- Logging secrets or personal data
- Calling basicConfig inside a library, which fights the application settings

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

Where should logging be configured?

Answer: Once, in the main program

What does log.exception add?

Answer: The traceback of the current exception

Which level fits a retry that will probably succeed?

Answer: WARNING

Why write JSON lines?

Answer: Log tools can search and aggregate fields without parsing free text

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Working with YAML, CSV and configuration files

12 minutes. Read and write the data formats of operations safely: YAML, CSV and INI, with validation and without surprises.

Infrastructure is described in files. Kubernetes manifests are YAML, exports from a spreadsheet are CSV, and older tools use INI. Python can read them all, but each has traps: YAML turns the word `no` into a boolean, CSV breaks on commas inside fields, and loading untrusted YAML the wrong way can run code.

What you will learn

- Parse YAML with safe_load and understand type surprises
- Use the csv module instead of splitting on commas
- Read INI-style files with configparser
- Validate what you read before you use it

YAML: always safe_load

YAML is not in the standard library; install PyYAML. Always use `safe_load`, which only builds plain data. The plain `load` function can create arbitrary Python objects from a crafted file.

```
import yaml

with open("deploy.yaml") as f:
    data = yaml.safe_load(f)

replicas = data["spec"]["replicas"]
```

Watch the type rules. Unquoted `no`, `off` and `yes` become booleans in YAML 1.1, so a country code `NO` can turn into `False`. A version such as `1.10` becomes the number 1.1. Quote strings that must stay text, and write files with `yaml.safe_dump(data, sort_keys=False)` to keep key order.

CSV: use the module

Splitting a line on commas breaks as soon as a value contains a comma or a quote. The `csv` module handles quoting for you.

```
import csv

with open("servers.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["cpu"])

with open("out.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "cpu"])
    w.writeheader()
    w.writerow({"name": "web, east", "cpu": 4})
```

Open CSV files with `newline=""`, as the documentation says, and convert numbers yourself, because every value is text. For big files, loop over rows instead of loading all of them into a list.

INI files with configparser

Many tools still use `[section]` files with `key = value` lines. `configparser` reads them and supports defaults and typed getters.

```
import configparser

cfg = configparser.ConfigParser()
cfg.read("app.ini")
port = cfg.getint("server", "port", fallback=8080)
debug = cfg.getboolean("server", "debug", fallback=False)
```

Keys are case-insensitive by default, and `%` has special meaning in values unless you use the raw parser.

MARGA by EknathaLabs marga.eknathalabs.com

Validate before you trust

A file that parses is not necessarily valid. Check required keys, types and ranges at the point where you read, and fail with a message that names the file and the key.

```
def require(data, key, kind):
    if key not in data:
        raise ValueError(f"deploy.yaml: missing key {key!r}")
    if not isinstance(data[key], kind):
        raise ValueError(f"deploy.yaml: {key!r} must be {kind.__name__}")
    return data[key]
```

For larger schemas, a library such as `jsonschema` or `pydantic` describes the rules in one place and reports every error at once.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Working with YAML, CSV and configuration files fits together

Part	What it does	What goes wrong
YAML: always safe_load	YAML is not in the standard library; install PyYAML.	Using yaml.load on files you did not write.
CSV: use the module	Splitting a line on commas breaks as soon as a value contains a comma or a quote.	Splitting CSV lines with split(",").
INI files with configparser	Many tools still use [section] files with key = value lines.	Forgetting newline="" when opening CSV files.
Validate before you trust	A file that parses is not necessarily valid.	Trusting parsed data without checking required keys.

Flow: YAML: always safe_load to CSV: use the module / CSV: use the module to INI files with configparser / INI files with configparser to Validate before you trust

Incident: The list that lost a server

> **Incident:** *A script that read an inventory exported from a spreadsheet silently dropped one server from a maintenance run.*

```
$ grep -n "db,east" inventory.csv
17:db,east,4,16
$ python3 -c "print('db,east,4,16'.split(','))"
['db', 'east', '4', '16']
```

The exporter wrote the name as `"db, east"` with quotes, and the script split lines on commas, so the columns shifted and the host failed a numeric check and was skipped. Switching to `csv.DictReader` fixed it. The team also added a check that compares the number of rows read with the number of lines in the file and stops on a mismatch.

Common mistakes

- Using yaml.load on files you did not write
- Splitting CSV lines with split(",")
- Forgetting newline="" when opening CSV files
- Trusting parsed data without checking required keys

Check yourself

Which function loads YAML without running arbitrary code?

Answer: yaml.safe_load

Why use the csv module?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: It handles quoted fields, commas and line breaks inside values

What does configparser.getint return when the key is missing and a fallback is given?

Answer: The fallback value

What should you do after parsing a config file?

Answer: Validate required keys, types and ranges and report errors that name the file

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Debugging Python scripts and services

15 minutes. Read tracebacks from the bottom, use logging and the debugger well, and find scripts that hang or leak using faulthandler and py-spy.

Python usually tells you what went wrong, and in which line. Many hours are lost because people read the top of the error instead of the bottom, add print statements everywhere, or cannot see why a script simply hangs. This lesson shows a calm routine for finding the cause.

What you will learn

- Reading a traceback correctly
- Using `logging` instead of `print`
- Stepping through code with `breakpoint()` and `pdb`
- Finding a script that hangs, with `faulthandler` and `py-spy`
- Reproducing a failure with the smallest possible input

Read the traceback from the bottom

```
$ python sync.py
Traceback (most recent call last):
  File "sync.py", line 31, in <module>
    main()
  File "sync.py", line 24, in main
    total += item["price"] * item["qty"]
KeyError: 'qty'
```

The last line names the error and the value (`KeyError: 'qty'`). The line above it shows where it happened. The frames above that show how the code got there. Here an item has no `qty`. Decide whether the data is wrong or the code should cope (`item.get("qty", 0)`), and then add a test with that input.

Logging beats print

```
import logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(name)s %(message)s")
log = logging.getLogger("sync")

log.info("fetching %s", url)
try:
    resp = session.get(url, timeout=10)
    resp.raise_for_status()
except Exception:
    log.exception("fetch failed for %s", url)   # includes the traceback
    raise
```

Levels let you keep debug detail switched off in normal runs. `log.exception` records the full traceback. Log identifiers (request id, file name), never passwords or tokens.

Stop and look: the debugger

```
def total(items):
    breakpoint()          # opens pdb here (python -m pdb script.py also works)
    return sum(i["price"] * i["qty"] for i in items)
```

At the `(Pdb)` prompt: `n` next line, `s` step into a call, `c` continue, `p expr` print a value, `l` list code, `w` show the stack, `q` quit. With pytest, `pytest -x --pdb` opens the debugger at the first failing test.

When a script hangs

```
import faulthandler, signal
faulthandler.register(signal.SIGUSR1)       # kill -USR1 PID prints every thread's stack
faulthandler.dump_traceback_later(300)      # or dump after 5 minutes

$ py-spy dump --pid 4321
```

MARGA by EknathaLabs marga.eknathalabs.com

```
Thread 4321 (idle): "MainThread"
    recv (socket.py:707)
    ...
    fetch (sync.py:18)
```

A stack that ends in `recv` for a long time means the program is waiting on the network. The usual cause is an HTTP call with no timeout. `py-spy` can inspect a running process without changing it.

Shrink the problem

Copy the failing input into a small test. Remove parts until the failure disappears, and the last part removed is the cause. A failing test that you keep is the best record of the bug.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Debugging Python scripts and services fits together

Part	What it does	What goes wrong
Read the traceback from the bottom	The last line names the error and the value (KeyError: 'qty').	Debugging with the full data set instead of a small failing example.
Logging beats print	Levels let you keep debug detail switched off in normal runs.	Catching Exception and doing nothing, which hides the real error.
Stop and look: the debugger	At the (Pdb) prompt: n next line, s step into a call, c continue, p expr print a value, l list code, w show the stack, q quit.	Check the Common mistakes list below this diagram for what usually goes wrong here.
When a script hangs	A stack that ends in recv for a long time means the program is waiting on the network.	Mutable default arguments.
Shrink the problem	Copy the failing input into a small test.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Read the traceback from the bottom to Logging beats print / Logging beats print to Stop and look: the debugger / Stop and look: the debugger to When a script hangs / When a script hangs to Shrink the problem

Incident: the job that never finished

> **Incident:** *A nightly sync usually takes ten minutes. One night it is still running at noon and blocks the next run.*

```
$ py-spy dump --pid 18822 | head -8
Thread 18822 (idle): "MainThread"
    recv (ssl.py:1134)
    read (http/client.py:466)
    send (requests/adapters.py:667)
    get (requests/sessions.py:602)
    fetch_page (sync.py:41)
```

The dump showed the program stuck reading from a socket in `fetch_page`. The request had no timeout, and the remote server had stopped responding. The fix was `timeout=(3, 15)`, a retry with a limit, and a lock file with a maximum run time.

Common mistakes

- Catching `Exception` and doing nothing, which hides the real error
- Printing secrets while debugging
- Debugging with the full data set instead of a small failing example
- Mutable default arguments

MARGA by EknathaLabs marga.eknathalabs.com

Check yourself

In a Python traceback, where do you usually find the error type and message?

Answer: On the last line

Why use logging instead of print for diagnostics?

Answer: It has levels, timestamps, destinations and can include tracebacks

A script hangs. Which tool shows where a running process is stuck?

Answer: py-spy dump or faulthandler

What is the usual cause of an HTTP call that waits forever?

Answer: No timeout was set on the request

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. SRE automation: AWS with boto3, safely

13 minutes. Write boto3 scripts that paginate, filter by tags, default to a dry run and use least-privilege roles, so automation helps and never surprises.

Scripts that talk to a cloud account can delete real things. The difference between a helpful script and an outage is a few habits: listing completely, selecting narrowly, previewing first and running with the least permission needed.

What you will learn

- Create sessions and clients with the right profile, role and region
- Use paginators so you never miss resources beyond the first page
- Select by tags and keep a dry-run default
- Handle errors from the API cleanly

Sessions, roles and regions

Never hard-code keys. Use a named profile or an assumed role, and set the region explicitly, since a script that quietly uses the default region will inspect the wrong place.

```
import boto3

def session(profile=None, region="eu-west-1"):
    return boto3.Session(profile_name=profile, region_name=region)

ec2 = session().client("ec2")
sts = session().client("sts")
print(sts.get_caller_identity()["Arn"])
```

Print the caller identity at the start of a risky script. It is a cheap guard against running against production by mistake.

Always paginate

API calls return limited pages. A loop that reads only the first response will miss resources and make your report wrong.

```
def untagged_volumes(ec2):
    paginator = ec2.get_paginator("describe_volumes")
    for page in paginator.paginate(Filters=[{"Name": "status", "Values": ["available"]}]):
        for vol in page["Volumes"]:
            tags = {t["Key"]: t["Value"] for t in vol.get("Tags", [])}
            if "owner" not in tags:
                yield vol["VolumeId"], vol["Size"]
```

Server-side filters reduce data; tags decide ownership. Treat a missing tag as "unknown owner", which is a reason to ask, not a reason to delete.

Dry run first

```
import botocore.exceptions as bx

def delete_volumes(ec2, ids, apply=False):
    for vid in ids:
        try:
            ec2.delete_volume(VolumeId=vid, DryRun=not apply)
        except bx.ClientError as err:
            code = err.response["Error"]["Code"]
            if code == "DryRunOperation":
                print("would delete", vid)
            else:
                print("failed", vid, code)
```

MARGA by EknathaLabs marga.eknathalabs.com

Many EC2 calls support `DryRun`, which returns the error `DryRunOperation` when the call would have succeeded. For services without it, implement your own: print the plan by default and require an explicit `--apply` flag. Catch `ClientError` and look at the error code, because a throttling or permission error needs different handling from a missing resource.

Least privilege

Give automation its own role that can do only the listed actions in the accounts it needs, and no more. A read-only role for reporting scripts and a separate, narrower one for changes makes accidents smaller. Log every change with the caller identity, the resource and the time.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How SRE automation: AWS with boto3, safely fits together

Part	What it does	What goes wrong
Sessions, roles and regions	Never hard-code keys. Use a named profile or an assumed role, and set the region explicitly, since a script that quietly uses the default region will inspect the wrong...	Relying on the default region or credentials.
Always paginate	API calls return limited pages. A loop that reads only the first response will miss resources and make your report wrong.	Reading only the first page of results.
Dry run first	Many EC2 calls support DryRun, which returns the error DryRunOperation when the call would have succeeded.	Deleting on "no owner tag" without asking.
Least privilege	Give automation its own role that can do only the listed actions in the accounts it needs, and no more.	Using one powerful role for everything.

Flow: Sessions, roles and regions to Always paginate / Always paginate to Dry run first / Dry run first to Least privilege

Incident: The cleanup that found nothing and then too much

> **Incident:** *A script meant to delete unattached volumes reported "0 candidates" in the test account and deleted volumes in production.*

```
$ python cleanup.py --region eu-west-1
caller: arn:aws:iam::111122223333:role/ops-admin
deleting 14 volumes
```

The test run had used the default region, which had no volumes. In production the region was set, the role was an admin role, and the script had no dry run. Some volumes were detached snapshots-in-waiting for a migration. The team restored from snapshots, changed the script to print the plan by default, require `--apply`, check an `owner` tag and run with a narrow role, and added the caller identity and region to the first line of output.

Common mistakes

- Reading only the first page of results
- Relying on the default region or credentials
- Deleting on "no owner tag" without asking
- Using one powerful role for everything

Check yourself

Why use a paginator?

Answer: API responses are limited in size, so you must read every page

MARGA by EknathaLabs marga.eknathalabs.com

What does a DryRunOperation error mean for an EC2 call made with DryRun?

Answer: The call would have succeeded

What is a safe default for a destructive script?

Answer: Print the plan and require an explicit apply flag

Why print the caller identity at the start?

Answer: To confirm which account and role the script is using

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. SRE automation: Kubernetes client scripts and reports

12 minutes. Use the official Python client to list workloads, find unhealthy pods and missing limits, and produce reports without scripting kubectl output.

`kubectl` is perfect for people. For scheduled reports and safe automation, a Python script using the API client gives you structured data, real error handling and tests. This lesson builds a small report you could run daily.

What you will learn

- Load cluster configuration inside and outside the cluster
- List pods and read their status as structured objects
- Find common problems: restarts, missing limits and pending pods
- Keep scripts read-only unless a change is explicitly requested

Connecting

```
from kubernetes import client, config

def api():
    try:
        config.load_incluster_config()
    except config.ConfigException:
        config.load_kube_config()
    return client.CoreV1Api()

v1 = api()
```

Inside a pod the client uses the service account token. Outside it reads your kubeconfig. Print the current context at start in laptop runs, since a script that changes the wrong cluster is a common accident.

A health report

```
def restarts(v1, namespace=None, threshold=5):
    pods = (v1.list_namespaced_pod(namespace) if namespace
            else v1.list_pod_for_all_namespaces())
    for p in pods.items:
        for cs in (p.status.container_statuses or []):
            if cs.restart_count >= threshold:
                reason = cs.last_state.terminated.reason if cs.last_state.terminated else ""
                yield p.metadata.namespace, p.metadata.name, cs.name, cs.restart_count, reason
```

The objects mirror the API: `metadata`, `spec`, `status`. Container statuses give restart counts and the reason for the last termination, such as `OOMKilled`. For large clusters pass `limit` and follow the `continue` token to page through results instead of loading everything at once.

Policy checks

```
def missing_limits(v1):
    for p in v1.list_pod_for_all_namespaces().items:
        for c in p.spec.containers:
            res = c.resources
            if not res or not res.limits or "memory" not in res.limits:
                yield p.metadata.namespace, p.metadata.name, c.name
```

Reports like this find pods without memory limits, images using `latest`, or pods pending for a long time. Write results as CSV or JSON so other tools can use them, and keep a stable sort order so day-to-day diffs are meaningful.

MARGA by EknathaLabs marga.eknathalabs.com

Read-only by default

Give the service account only `get`, `list` and `watch` permissions on the resources it reports on. If a script must change something (for example cordon a node), ask for a flag, log what it changed, and use a separate, narrower role. Handle `ApiException`: a `403` means a permissions gap, a `404` often means someone already deleted the object, and a `429` or `5xx` should be retried with backoff.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How SRE automation: Kubernetes client scripts and... fits together

Part	What it does	What goes wrong
Connecting	Inside a pod the client uses the service account token.	Running with cluster-admin for a read-only report.
A health report	The objects mirror the API: metadata, spec, status.	Loading huge lists in one request.
Policy checks	Reports like this find pods without memory limits, images using latest, or pods pending for a long time.	Policy checks Reports like this find pods without memory limits, images using latest, or pods pending for a long time.
Read-only by default	Give the service account only get, list and watch permissions on the resources it reports on.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Connecting to A health report / A health report to Policy checks / Policy checks to Read-only by default

Incident: The report that missed the problem

Incident: *A daily report said "no restarts above threshold", yet a service was crash-looping all week.*

```
$ python report.py --threshold 5
0 containers above threshold
$ kubectl get pods -n shop
checkout-7c9   0/1   CrashLoopBackOff   2 (30s ago)   6m
```

The pod was recreated by its controller every few minutes after failing readiness, so the restart count of each new pod stayed low. The report counted restarts per pod, not per workload. The team changed it to also group by owner and report the number of distinct pods created in the last day, and to flag any workload whose pods are not ready. The count of restarts alone was the wrong signal.

Common mistakes

- Parsing `kubectl` text output in scripts
- Running with cluster-admin for a read-only report
- Loading huge lists in one request
- Measuring a symptom at the wrong level (pod instead of workload)

Check yourself

How does the Python client authenticate inside a pod?

Answer: With the service account token via load_incluster_config

What does a 403 ApiException usually mean?

Answer: The service account lacks permission for that call

How do you handle very large list calls?

Answer: Use limit and the continue token, or filter by namespace or label

Why group restarts by workload as well as by pod?

Answer: Replacement pods reset the per-pod count and hide a crash loop

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 13. SRE automation: retries, backoff, timeouts and idempotency

12 minutes. Make scripts survive flaky networks and APIs with timeouts, exponential backoff with jitter and idempotent operations, without causing retry storms.

Networks fail, APIs throttle and servers restart. A script that calls the network without a timeout can hang forever, and a script that retries blindly can make an outage worse. Good automation is patient, bounded and safe to run twice.

What you will learn

- Set timeouts on every network call
- Retry only what is safe, with exponential backoff and jitter
- Make operations idempotent so a retry cannot double the effect
- Know when to stop and report

Timeouts everywhere

Most HTTP libraries have no default timeout. Always set one, and separate connect from read time.

```
import requests

resp = requests.get("https://api.example.com/v1/items", timeout=(3.05, 10))
resp.raise_for_status()
```

A timeout turns a silent hang into an error you can handle. Choose values from the real latency of the service, not from guesses, and keep an overall deadline for the whole job.

Backoff with jitter

Retry transient failures only: connection errors, timeouts, `429` and `5xx`. Never retry `400`, `401`, `403` or `404` blindly, since the answer will not change. Wait longer after each attempt, and add randomness so many clients do not retry at the same moment.

```
import random, time

def call_with_retry(fn, attempts=5, base=0.5, cap=20):
    for n in range(attempts):
        try:
            return fn()
        except (requests.ConnectionError, requests.Timeout) as err:
            if n == attempts - 1:
                raise
            delay = random.uniform(0, min(cap, base * 2 ** n))
            time.sleep(delay)
```

This is "full jitter": a random delay between zero and a growing limit. If a server sends a `Retry-After` header, obey it. Put a cap on total attempts and total time, and log each retry with its reason so you can see a pattern.

Idempotency

An operation is idempotent when running it twice has the same effect as running it once. "Set replicas to 3" is idempotent. "Add one replica" and "create an order" are not. For non-idempotent calls, send an **idempotency key** (many APIs support one), check whether the thing already exists before creating it, or use "create if not exists" semantics. Then a timeout, where you do not know if the first call worked, is safe to retry.

MARGA by EknathaLabs
marga.eknathalabs.com

Circuit breaking and giving up

If a dependency is down, hammering it prevents recovery. Stop after a bounded number of failures, wait, and report clearly what was done and what was not. A script that finishes half the work should say which half, so a person or the next run can continue.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How SRE automation: retries, backoff, timeouts and... fits together

Part	What it does	What goes wrong
Timeouts everywhere	Most HTTP libraries have no default timeout.	Calls with no timeout.
Backoff with jitter	Retry transient failures only: connection errors, timeouts, 429 and 5xx.	Retrying errors that can never succeed.
Idempotency	An operation is idempotent when running it twice has the same effect as running it once.	Retrying non-idempotent writes with no key.
Circuit breaking and giving up	If a dependency is down, hammering it prevents recovery.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Timeouts everywhere to Backoff with jitter / Backoff with jitter to Idempotency / Idempotency to Circuit breaking and giving up

Incident: The retry storm

> **Incident:** *A short database failover turned into a 40-minute outage because every client retried immediately and in a tight loop.*

```
$ grep -c "retry" /var/log/app/client.log
184220
$ grep "attempt" /var/log/app/client.log | head -2
attempt 1 failed, retrying immediately
attempt 2 failed, retrying immediately
```

The failover took 20 seconds, but thousands of clients retried without delay and without limit, so the new primary was flooded as soon as it started and fell over again. The team added exponential backoff with jitter and a retry budget, put timeouts on every call, and made the job's writes idempotent so retries were safe. The next failover passed unnoticed.

Common mistakes

- Calls with no timeout
- Retrying immediately or without a limit
- Retrying errors that can never succeed
- Retrying non-idempotent writes with no key

Check yourself

Why add jitter to backoff?

Answer: So clients do not all retry at the same moment

Which status codes are usually worth retrying?

Answer: 429 and 5xx, plus connection errors and timeouts

What makes an operation idempotent?

Answer: Running it twice gives the same result as once

A request times out and you do not know if it worked. What makes a retry safe?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: An idempotency key or an existence check

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 14. SRE automation: parsing logs and building incident timelines

12 minutes. Parse structured and plain logs with Python, count and group errors, and merge events from several sources into one sorted timeline for an incident review.

During an incident the evidence is spread across application logs, load balancer logs, deploy history and alerts. Python can collect it into one timeline in minutes, and the same script can be reused for the next post-mortem.

What you will learn

- Parse JSON lines and plain text lines safely
- Count and group by status, endpoint or time bucket
- Normalize time zones and merge several sources
- Stream large files without loading them into memory

Read and parse safely

```
import json, re
from datetime import datetime, timezone

LINE = re.compile(r'^(?P<ts>\S+) (?P<level>[A-Z]+) (?P<msg>.*)$')

def parse(line):
    line = line.strip()
    if not line:
        return None
    if line.startswith("{"):
        try:
            d = json.loads(line)
            return {"ts": d.get("ts"), "level": d.get("level", ""), "msg": d.get("msg", "")}
        except json.JSONDecodeError:
            return None
    m = LINE.match(line)
    return m.groupdict() if m else None
```

Logs are messy: partial lines, mixed formats, binary noise. Return `None` for lines you cannot parse and count them, instead of crashing. If many lines fail, the format has changed, which is itself a finding.

Count and group

```
from collections import Counter

def top_errors(path, n=5):
    c = Counter()
    with open(path, errors="replace") as fh:
        for line in fh:
            e = parse(line)
            if e and e["level"] == "ERROR":
                c[re.sub(r"\d+", "N", e["msg"])[:80]] += 1
    return c.most_common(n)
```

Replace numbers and ids with a placeholder so that similar messages group together. Iterating the file line by line streams it, so memory stays small even for gigabytes. For time buckets, truncate timestamps to the minute and count per bucket to see when the rate changed.

MARGA by EknathaLabs marga.eknathalabs.com

One timeline

```
def to_utc(text):
    dt = datetime.fromisoformat(text.replace("Z", "+00:00"))
    return dt.astimezone(timezone.utc)

def merge(sources):
    events = []
    for name, rows in sources.items():
        for ts, text in rows:
            events.append((to_utc(ts), name, text))
    return sorted(events)
```

Convert every timestamp to UTC before sorting; mixed time zones are the most common reason timelines look wrong. Combine app errors, load balancer 5xx counts, deploy markers and alert times, and mark the moments of detection, mitigation and recovery. The output (a sorted list or a CSV) is the factual base of the post-mortem.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How SRE automation: parsing logs and building... fits together

Part	What it does	What goes wrong
Read and parse safely	Logs are messy: partial lines, mixed formats, binary noise.	Crashing on one malformed line.
Count and group	Replace numbers and ids with a placeholder so that similar messages group together.	Sorting events with mixed time zones.
One timeline	Convert every timestamp to UTC before sorting; mixed time zones are the most common reason timelines look wrong.	Hand-copying times into a document instead of generating them.

Flow: Read and parse safely to Count and group / Count and group to One timeline

Incident: The timeline that blamed the wrong release

Incident: *A post-mortem draft said errors started 10 minutes before the deploy, so the team nearly ruled out the release.*

```
$ head -1 app.log lb.log
==> app.log <==
2026-03-04T14:02:11+00:00 ERROR upstream timeout
==> lb.log <==
2026-03-04 19:32:40 status=502
```

The load balancer wrote local time (UTC plus 5:30) without an offset. After converting both to UTC, the first 502 came 30 seconds after the deploy finished, not before. The team added a timeline script that requires explicit offsets and marks the source of each event, and the review then focused on the release's connection pool change.

Common mistakes

- Sorting events with mixed time zones
- Loading a huge file into memory
- Crashing on one malformed line
- Hand-copying times into a document instead of generating them

Check yourself

How should you handle a log line that cannot be parsed?

Answer: Skip it, count it and continue

MARGA by EknathaLabs marga.eknathalabs.com

Why replace numbers with a placeholder before counting messages?

Answer: So similar messages group into one pattern

What must you do before merging events from several sources?

Answer: Convert all timestamps to a single time zone such as UTC

How do you process a 10 GB log with little memory?

Answer: Iterate line by line instead of reading it all

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 15. SRE automation: chatops, alert handlers and webhooks

12 minutes. Build a small webhook receiver that verifies signatures, deduplicates alerts and posts useful messages, and keep automated actions safe with allow lists and approvals.

Many teams connect alerts, chat and automation: an alert posts to a channel, and a button or command runs a runbook step. The idea is simple, but the receiver is an internet-facing program that can run actions, so it needs care.

What you will learn

- Receive a webhook and verify that it is genuine
- Deduplicate and group alerts so channels stay readable
- Post structured messages with context and links
- Keep actions behind allow lists, approval and an audit trail

A minimal receiver with verification

```
import hmac, hashlib, json
from http.server import BaseHTTPRequestHandler, HTTPServer

SECRET = b"read-this-from-a-secret-store"

class Hook(BaseHTTPRequestHandler):
    def do_POST(self):
        size = int(self.headers.get("Content-Length", 0))
        body = self.rfile.read(min(size, 1_000_000))
        sent = self.headers.get("X-Signature", "")
        want = hmac.new(SECRET, body, hashlib.sha256).hexdigest()
        if not hmac.compare_digest(sent, want):
            self.send_response(401); self.end_headers(); return
        handle(json.loads(body))
        self.send_response(202); self.end_headers()

def handle(event):
    print("alert", event.get("name"), event.get("severity"))

if __name__ == "__main__":
    HTTPServer(("127.0.0.1", 8080), Hook).serve_forever()
```

A webhook URL can leak, so never trust the body alone. The sender signs the body with a shared secret, and you recompute the HMAC and compare in constant time with `hmac.compare_digest`. Limit the body size, answer quickly with `202`, and do slow work in a queue or a thread. In production use a real framework behind TLS, with the secret in a secret manager.

Keep the channel readable

Send one message per incident, not per alert. Use a **fingerprint** (service, alert name and environment) as the key. If it was seen in the last few minutes, update the existing message or increase a counter instead of posting again. Include what matters: severity, service, a one-line symptom, a dashboard link, the runbook link and who is on call. Avoid walls of JSON.

Actions with guard rails

A button that restarts a service is useful and dangerous. Define an **allow list** of actions (restart service X in staging, scale Y between 2 and 10), check who pressed the button against a group, ask for a second approval for production, and write an audit line with person, action, target and result. Make actions idempotent and time-limited, and never let chat text become a shell command: map a fixed action name to a fixed function.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How SRE automation: chatops, alert handlers and... fits together

Part	What it does	What goes wrong
A minimal receiver with verification	A webhook URL can leak, so never trust the body alone.	Accepting webhooks without a signature check.
Keep the channel readable	Send one message per incident, not per alert.	Posting every alert as a separate message.
Actions with guard rails	A button that restarts a service is useful and dangerous.	Actions without approval or an audit trail.

Flow: A minimal receiver with verification to Keep the channel readable / Keep the channel readable to Actions with guard rails

Incident: The channel nobody read

> **Incident:** *During a real outage the on-call engineer missed the key alert because the channel had 900 messages in an hour.*

```
$ grep -c "DiskPressure" alerts.log
312
$ grep -c "PaymentsDown" alerts.log
1
```

One flapping node alert posted 312 times and buried the single message that mattered. The team added fingerprint-based grouping with a cooldown, routed severity-high alerts to paging and the rest to a quiet channel, and added a rule that a flapping alert posts once with a counter. The next incident had a short, readable timeline.

Common mistakes

- Accepting webhooks without a signature check
- Posting every alert as a separate message
- Letting chat input reach a shell
- Actions without approval or an audit trail

Check yourself

How do you confirm that a webhook is genuine?

Answer: Recompute the HMAC of the body with the shared secret and compare in constant time

What should a deduplication key contain?

Answer: Service, alert name and environment

Why use an allow list of actions?

Answer: So chat users can run only approved, bounded operations

Why respond quickly with 202?

Answer: The sender should not time out while you do slow work elsewhere

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 16. SRE automation: safe bulk operations and audit trails

12 minutes. Run changes across many hosts or resources with a plan, batches, rate limits, a kill switch, resumable state and a clear audit trail.

Running a command on one server is a task; running it on 800 servers is an operation. The risks multiply: a mistake repeats everywhere, the target may be overloaded, and half-finished work is hard to describe. Bulk automation needs a small set of safety features built in from the start.

What you will learn

- Build a plan and review it before any change
- Work in small batches with a pause and a health check
- Respect rate limits and concurrency limits
- Keep state so a run can resume, and write an audit trail

Plan, then apply

Separate the decision of what to change from the act of changing. The plan step reads current state and produces a list of exact actions. A person (or a check) reviews the list, and only then does the apply step run it. If state changes between plan and apply, re-plan.

```
import json, time

def make_plan(hosts, wanted):
    return [{"host": h["name"], "from": h["version"], "to": wanted}
            for h in hosts if h["version"] != wanted]

def save_plan(plan, path="plan.json"):
    with open(path, "w") as fh:
        json.dump(plan, fh, indent=2)
    print(len(plan), "changes planned, review", path)
```

Printing the count is a cheap sanity check: a plan that touches 780 of 800 hosts when you expected 12 should stop you.

Batches, limits and a health gate

```
from concurrent.futures import ThreadPoolExecutor

def run(plan, apply_one, healthy, batch=10, workers=3, pause=30):
    done = []
    for i in range(0, len(plan), batch):
        chunk = plan[i:i + batch]
        with ThreadPoolExecutor(max_workers=workers) as pool:
            results = list(pool.map(apply_one, chunk))
        done.extend(results)
        if not healthy():
            print("health check failed, stopping after", len(done))
            break
        time.sleep(pause)
    return done
```

Start with one canary target, then a small batch, then larger ones. Limit workers to what the target systems and APIs can take, and stop on the first failed health check. Add a **kill switch**: a file or flag the script checks between batches, so any person can halt the run without killing processes.

Resume and audit

Record each result as it happens, in an append-only file, with time, operator, target, action and outcome. On restart, skip targets already marked done, so a run that died halfway continues without repeating work. Make each action idempotent in case the last one was cut off. After the run, print a summary: planned, succeeded, failed, skipped, with the failures listed.

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How SRE automation: safe bulk operations and audit... fits together

Part	What it does	What goes wrong
Plan, then apply	Separate the decision of what to change from the act of changing.	No canary, no health check and no stop button.
Batches, limits and a health gate	Start with one canary target, then a small batch, then larger ones.	No record of who changed what and when.
Resume and audit	Record each result as it happens, in an append-only file, with time, operator, target, action and outcome.	Hidden state that makes reruns repeat work.

Flow: Plan, then apply to Batches, limits and a health gate / Batches, limits and a health gate to Resume and audit

Incident: The change that went everywhere

Incident: *A script that updated an agent configuration on all hosts applied a bad value to 600 machines in four minutes.*

```
$ python rollout.py --all --workers 50
applied 600/800 ... monitoring: errors rising
$ ls kill-switch
ls: cannot access 'kill-switch': No such file or directory
```

There was no plan review, no canary and no health gate, and 50 workers worked with no pause, so the whole fleet was affected before anyone saw the first alert. Recovery meant a rollback script written during the incident. The team changed the tool to require a plan file, start with one canary, run in batches of 20 with a health check between them and a kill switch, and to log every host change to an audit file.

Common mistakes

- Applying to everything at once
- No canary, no health check and no stop button
- Hidden state that makes reruns repeat work
- No record of who changed what and when

Check yourself

Why separate plan from apply?

Answer: A person or check can review the exact changes before anything happens

What is the purpose of a canary?

Answer: Test the change on one small target before the rest

What makes a long run resumable?

Answer: Recorded state of finished targets and idempotent actions

What should an audit record contain?

Answer: Time, operator, target, action and result

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Python

Environments, core syntax, files, subprocess, HTTP, JSON, cloud SDK, testing and packaging snippets for automation.

Environments

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install requests==2.32.3
python -m pip freeze > requirements.txt
python -m pip install -r requirements.txt
python -m pip list --outdated
deactivate
```

One virtual environment per project. Call `python -m pip` so pip matches the interpreter.

Core syntax

```
servers = ["web-1", "web-2", "db-1"]
limits = {"cpu": 80, "memory": 90}
web = [s for s in servers if s.startswith("web")]
by_len = {s: len(s) for s in servers}
print(f"{len(web)} web servers, cpu limit {limits.get('cpu', 100)}")

def over(metric: str, value: float) -> bool:
    return value > limits.get(metric, 100)

if __name__ == "__main__":
    raise SystemExit(0)
```

Files and paths

```
from pathlib import Path

p = Path("/etc/myapp/app.conf")
text = p.read_text(encoding="utf-8")
Path("/var/tmp/out").mkdir(parents=True, exist_ok=True)
for f in Path("/var/log/myapp").glob("*.log"):
    print(f.name, f.stat().st_size)
with open("report.txt", "w", encoding="utf-8") as fh:
    fh.write("done\n")
```

Run commands

```
import subprocess

r = subprocess.run(["systemctl", "is-active", "nginx"], capture_output=True, text=True, timeout=10 \
    )
print(r.returncode, r.stdout.strip())
subprocess.run(["tar", "-czf", "/tmp/site.tgz", "/var/www"], check=True)
```

Pass a list, avoid `shell=True`, always set a timeout.

JSON and HTTP

```
import json
import os
import requests

data = json.loads('{"name": "web-1", "cpu": 0.4}')
print(json.dumps(data, indent=2, sort_keys=True))

resp = requests.get(
    "https://api.example.com/v1/servers",
    headers={"Authorization": "Bearer " + os.environ["API_TOKEN"]},
    params={"limit": 50},
    timeout=10,
)
resp.raise_for_status()
items = resp.json()["items"]
```

Always set `timeout`, call `raise_for_status`, read tokens from the environment.

MARGA by EknathaLabs
marga.eknathalabs.com

Errors and logging

```
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("job")
try:
    risky()
except ValueError as exc:
    log.exception("failed: %s", exc)
    raise
```

AWS and Azure SDKs

```
import boto3

print(boto3.client("sts").get_caller_identity()["Arn"])
ec2 = boto3.client("ec2", region_name="eu-west-1")
for page in ec2.get_paginator("describe_instances").paginate():
    for res in page["Reservations"]:
        for inst in res["Instances"]:
            print(inst["InstanceId"])

from azure.identity import DefaultAzureCredential
from azure.mgmt.compute import ComputeManagementClient

compute = ComputeManagementClient(DefaultAzureCredential(), "SUBSCRIPTION_ID")
for vm in compute.virtual_machines.list_all():
    print(vm.name)
```

CLI, tests, packaging

```
import argparse

p = argparse.ArgumentParser(prog="tool")
p.add_argument("paths", nargs="+")
p.add_argument("--threshold", type=int, default=85)
args = p.parse_args()
```

```
python -m pip install pytest
python -m pytest -q
python -m pytest -q -k "threshold" -x
python -m pip install .
python -m pip install pipx
```

Mock slow or risky calls with `unittest.mock.patch`; use the `tmp_path` fixture for files.

Environments and packages

```
python3 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pip install --upgrade pip
python -m pip freeze > requirements.lock
python -m pip list --outdated
python -m pip check
python -m pip install -e .
python -m pip install --require-hashes -r requirements.lock
python -m pip_audit
python -V; which python
```

> **Warning:** *Do not run* `sudo pip install`. *It changes system packages and can break the operating system tools. Use a virtual environment.*

> **Warning:** *Installing a package runs its code, and a mistyped name can be a malicious look-alike. Check the name and use a lock file with hashes in production.*

MARGA by EknathaLabs marga.eknathalabs.com

Debug and profile

```
python -X dev -W error script.py
python -m pdb script.py
python -m trace --trace script.py | head
python -m cProfile -o out.prof -s cumtime script.py
python -m timeit -s 'import json' 'json.dumps({"a": 1})'
python -m http.server 8000 --bind 127.0.0.1
python -m json.tool data.json
python -c 'import sys; print(sys.path)'
```

Warning: `python -m http.server` *serves the current directory to anyone who can reach the port. Bind to* `127.0.0.1`*, and do not run it in a folder with secrets.*

Warning: `breakpoint()` *and* `pdb` *left in code stop a service waiting for input. Remove them before you commit.*

Files, JSON and HTTP in scripts

```
from pathlib import Path
import json, os, subprocess, requests

data = json.loads(Path("config.json").read_text())
Path("out").mkdir(parents=True, exist_ok=True)
r = requests.get(url, timeout=5, headers={"Authorization": f"Bearer {os.environ['TOKEN']}"})
r.raise_for_status()
out = subprocess.run(["kubectl", "get", "pods", "-o", "json"], capture_output=True, text=True, che \
    ck=True, timeout=30)
pods = json.loads(out.stdout)["items"]
```

Warning: `subprocess.run(cmd, shell=True)` *with user input allows command injection. Pass a list of arguments, and keep* `shell=False`*.*

Warning: `requests.get` *without* `timeout` *can hang forever. Always set one, and never disable certificate checks with* `verify=False`*.*

Warning: `pickle.load`*,* `yaml.load` *and* `eval` *on untrusted data can run code. Use* `json` *and* `yaml.safe_load`*.*

Warning: *Do not put tokens or passwords in the code or in logs. Read them from environment variables or a secrets manager.*

Testing and quality

```
python -m pytest -q -x --lf
python -m pytest --cov=app --cov-report=term-missing
python -m ruff check . && python -m ruff format --check .
python -m mypy app
python -m build
python -m twine check dist/*
```

Danger: `twine upload` *publishes to a public index and cannot be undone: a version number can never be reused. Test with the test index first.*

MARGA by EknathaLabs
marga.eknathalabs.com